

Reparto de Tareas

Space Invaders — Implementación de Patrones de Diseño

Equipo: Cachopín

Fecha: 15 de abril de 2026

Miembros del equipo

- **Miembro 1:** Unai Urrutia
- **Miembro 2:** Ekain Ortega
- **Miembro 3:** Eder Lorenzo
- **Miembro 4:** Asier López

Planificación inicial

Tareas previstas para la implementación de patrones de diseño y su tiempo estimado:

1. Implementar patrón Factory para creación de naves — [X] horas
2. Implementar patrón Strategy para tipos de disparo — [X] horas
3. Implementar patrón Composite para estructura jerárquica de píxeles — [X] horas
4. Actualizar diagrama de clases UML — [X] horas
5. Crear 3 tipos de naves diferentes (Nave1, Nave2, Nave3) — [X] horas
6. Implementar naves multipixel con geometría variable — [X] horas
7. Implementar enemigos multipixel con estructura Composite — [X] horas
8. Implementar tipos de disparo (Pixel, Flecha, Rombo) — [X] horas
9. Documentar patrones Factory, Strategy y Composite — [X] horas

Tiempo total estimado: [X] horas.

Resumen de tareas realizadas

Tareas efectivamente completadas y su tiempo real:

1. Implementar patrón Factory para creación de naves — [X] horas
2. Implementar patrón Strategy para tipos de disparo — [X] horas
3. Implementar patrón Composite para estructura jerárquica de píxeles — [X] horas
4. Actualizar diagrama de clases UML — [X] horas
5. Crear 3 tipos de naves diferentes (Nave1, Nave2, Nave3) — [X] horas
6. Implementar naves multipixel con geometría variable — [X] horas
7. Implementar enemigos multipixel con estructura Composite — [X] horas
8. Implementar tipos de disparo (Pixel, Flecha, Rombo) — [X] horas
9. Documentar patrones Factory, Strategy y Composite — [X] horas

Tiempo total real: [X] horas.

Detalle de tareas

Tarea 1 — Implementar patrón Factory para creación de naves

Descripción:

Desarrollo de la clase **NaveFactory** como punto único de creación de naves jugables. Implementación del patrón Singleton para garantizar una única instancia. El método **generate(String)** recibe un identificador y retorna la nave correspondiente con parámetros predefinidos.

Tiempo estimado:

[X] horas.

Tiempo real:

[X] horas.

Responsables y dedicación:

- Unai Urrutia — [X] horas
- Ekain Ortega — [X] horas
- Eder Lorenzo — 0 horas
- Asier López — [X] horas

Tarea 2 — Implementar patrón Strategy para tipos de disparo

Descripción:

Creación de la interfaz `StrategyDisparo` con métodos: `getTipo()`, `getMunicion()`, `gastar()`, `tieneMunicion()` y `crearDisparo()`. Implementación de tres estrategias concretas: `DisparoPixel` (munición infinita), `DisparoFlecha` (30 unidades) y `DisparoRombo` (20 unidades).

Tiempo estimado:

[X] horas.

Tiempo real:

[X] horas.

Responsables y dedicación:

- Unai Urrutia — [X] horas
- Ekain Ortega — [X] horas
- Eder Lorenzo — 0 horas
- Asier López — [X] horas

Tarea 3 — Implementar patrón Composite para estructura jerárquica de píxeles

Descripción:

Desarrollo de la interfaz `Component` que define el contrato común. Implementación de `Pixel` como hoja del `Composite` y `Composite` como contenedor de componentes. Diferenciación entre píxeles de nave (`proyectil=false`) y píxeles de disparo (`proyectil=true`) con comportamientos distintos. Implementación de métodos de movimiento validado, notificación de eventos y gestión de ciclo de vida.

Tiempo estimado:

[X] horas.

Tiempo real:

[X] horas.

Responsables y dedicación:

- Unai Urrutia — [X] horas
- Ekain Ortega — [X] horas
- Eder Lorenzo — 0 horas
- Asier López — [X] horas

Tarea 4 — Actualizar diagrama de clases UML

Descripción:

Elaboración del diagrama UML de clases que muestre las relaciones entre `NaveFactory`, `Naves` (clase abstracta), `Nave1`, `Nave2`, `Nave3`, `StrategyDisparo` y sus implementaciones, `Component`, `Pixel`, `Composite`, `Disparo`, `Espacio` y otros elementos del modelo. Se debe reflejar la jerarquía de herencia, las relaciones de composición y la implementación de interfaces.

Tiempo estimado:

[X] horas.

Tiempo real:

[X] horas.

Responsables y dedicación:

- Unai Urrutia — [X] horas
- Ekain Ortega — [X] horas
- Eder Lorenzo — 0 horas
- Asier López — [X] horas

Tarea 5 — Crear 3 tipos de naves diferentes (`Nave1`, `Nave2`, `Nave3`)

Descripción:

Desarrollo de las clases `Nave1`, `Nave2` y `Nave3` que extienden `Naves`. Cada nave tiene una combinación diferente de estrategias de disparo permitidas: `Nave1` (Pixel y Flecha), `Nave2` (Pixel y Rombo), `Nave3` (Pixel, Flecha y Rombo). Cada nave tiene geometría única y origen de disparo específico.

Tiempo estimado:

[X] horas.

Tiempo real:

[X] horas.

Responsables y dedicación:

- Unai Urrutia — [X] horas
- Ekain Ortega — [X] horas
- Eder Lorenzo — 0 horas
- Asier López — [X] horas

Tarea 6 — Implementar naves multipixel con geometría variable

Descripción:

Implementación del método `construir()` en cada nave específica. Cada nave está compuesta por un `Composite` raíz que contiene múltiples `Pixel` en posiciones específicas formando formas geométricas distintas. Nave1 forma una T invertida, Nave2 un rectángulo, Nave3 una T normal. Implementación de métodos como `celdasOcupadas()` para retornar la geometría de cada nave.

Tiempo estimado:

[X] horas.

Tiempo real:

[X] horas.

Responsables y dedicación:

- Unai Urrutia — [X] horas
- Ekain Ortega — [X] horas
- Eder Lorenzo — 0 horas
- Asier López — [X] horas

Tarea 7 — Implementar enemigos multipixel con estructura Composite

Descripción:

Desarrollo de la clase `Enemigo` usando el patrón Composite para crear enemigos compuestos por múltiples píxeles. Los enemigos tienen geometría multipixel similar a las naves. Implementación de métodos de movimiento, rotación y comportamiento táctico en flota. Integración con `FlotaEnemigos` para manejo coordinado de múltiples enemigos.

Tiempo estimado:

[X] horas.

Tiempo real:

[X] horas.

Responsables y dedicación:

- Unai Urrutia — [X] horas
- Ekain Ortega — [X] horas
- Eder Lorenzo — 0 horas
- Asier López — [X] horas

Tarea 8 — Implementar tipos de disparo (Pixel, Flecha, Rombo)

Descripción:

Implementación del método `crearDisparo()` en cada estrategia concreta. `DisparoPixel` crea un `Pixel` simple (1 celda). `DisparoFlecha` crea un `Composite` con 3 píxeles en forma de V invertida. `DisparoRombo` crea un `Composite` con 5 píxeles en forma de rombo. Cada disparo es un `Component` que se comporta como proyectil (notifica movimiento, se desactiva al salir del tablero).

Tiempo estimado:

[X] horas.

Tiempo real:

[X] horas.

Responsables y dedicación:

- Unai Urrutia — [X] horas
- Ekain Ortega — [X] horas
- Eder Lorenzo — 0 horas
- Asier López — [X] horas

Tarea 9 — Documentar patrones Factory, Strategy y Composite

Descripción:

Redacción de documentación completa en formato LaTeX describiendo la implementación de los tres patrones de diseño. Se incluye: descripción general de cada patrón, problema que resuelve, implementación en el proyecto, interfaces y clases concretas, diagramas de secuencia, flujos de ejecución, ventajas, y cómo trabajan juntos los patrones. Elaboración de diagramas conceptuales para ilustrar las relaciones entre componentes.

Tiempo estimado:

[X] horas.

Tiempo real:

[X] horas.

Responsables y dedicación:

- Unai Urrutia — [X] horas
- Ekain Ortega — [X] horas
- Eder Lorenzo — 0 horas
- Asier López — [X] horas

Notas y observaciones

- **Desviaciones significativas:** [Explica si el tiempo real fue muy diferente del estimado y por qué]
- **Problemas encontrados:** [Enumera los principales problemas y cómo se resolvieron]
- **Lecciones aprendidas:** [Qué aprendió el equipo durante la implementación de patrones]
- **Sugerencias para futuros proyectos:** [Mejoras propuestas sobre el uso de patrones]

Resumen estadístico

Miembro	Horas estimadas	Horas reales
Unai Urrutia	[X]	[X]
Ekain Ortega	[X]	[X]
Eder Lorenzo	0	0
Asier López	[X]	[X]
TOTAL	[X]	[X]